

Festival

A Nayra está num festival a jogar um jogo em que o grande prémio é uma viagem à Laguna Colorada. O jogo consiste em utilizar fichas para comprar cupões. A compra de um cupão pode dar origem a fichas adicionais. O objetivo é obter o maior número possível de cupões.

Ela começa o jogo com A fichas. Existem N cupões, numerados de 0 a $N - 1$. A Nayra tem de pagar $P[i]$ fichas ($0 \leq i < N$) para comprar o cupão i (e tem de ter pelo menos $P[i]$ fichas antes da compra). Ela pode comprar cada cupão no máximo uma vez.

Além disso, a cada cupão i ($0 \leq i < N$) é atribuído um **tipo**, denotado por $T[i]$, que é um número inteiro **entre 1 e 4, inclusive**. Depois de a Nayra comprar o cupão i , o número restante de fichas que tem é multiplicado por $T[i]$. Formalmente, se ela tiver X fichas numa determinada altura do jogo, e compra o cupão i (que requer $X \geq P[i]$), então ela terá $(X - P[i]) \cdot T[i]$ fichas após a compra.

A tua tarefa é determinar quais os cupões que a Nayra deve comprar e em que ordem, para maximizar o número total de **cupões** que ela tem no final. Se houver mais do que uma sequência de compras que atinja este objetivo, podes reportar qualquer uma delas.

Detalhes de Implementação

Deve implementar a seguinte função.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A o número inicial de fichas que a Nayra tem.
- P um vector de comprimento N que especifica os preços dos cupões.
- T um vector de comprimento N que especifica os tipos de cupões.
- Esta função é chamada exatamente uma vez para cada caso de teste.

A função deve devolver um vector R , que especifica as compras de Nayra da seguinte forma:

- O comprimento de R deve ser o número máximo de cupões que ela pode comprar.
- Os elementos do vector são os índices dos cupões que ela deve comprar, por ordem cronológica. Ou seja, ela compra primeiro o cupão $R[0]$, depois o cupão $R[1]$, e assim por diante.

- Todos os elementos de R devem ser únicos.

Se não for possível comprar cupões, R deve ser um conjunto vazio.

Restrições

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para cada i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para cada i tal que $0 \leq i < N$.

Subtarefas

Subtarefa	Pontuação	Restrições adicionais
1	5	$T[i] = 1$ para cada i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para cada i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra pode comprar todos os cupões N (em alguma ordem).
6	16	$(A - P[i]) \cdot T[i] < A$ para cada i tal que $0 \leq i < N$.
7	18	Nenhuma restrição adicional.

Exemplos

Exemplo 1

Considera a seguinte chamada.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Inicialmente, Nayra tem $A = 13$ fichas. Ela pode comprar 3 cupões pela ordem apresentada abaixo:

Cupão comprado	Preço do cupão	Tipo de cupão	Número de fichas após a compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Neste exemplo, não é possível que a Nayra compre mais do que 3 cupões, e a sequência de compras descrita acima é a única maneira de ela comprar 3 deles. Portanto, a função deve devolver $[2, 3, 0]$.

Exemplo 2

Considera a seguinte chamada.

```
max_coupons(9, [6, 5], [2, 3])
```

Neste exemplo, a Nayra pode comprar os dois cupões em qualquer ordem. Portanto, a função deve devolver $[0, 1]$ ou $[1, 0]$.

Exemplo 3

Considera a seguinte chamada.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Neste exemplo, a Nayra tem uma ficha, que não é suficiente para comprar cupões. Portanto, a função deve devolver $[]$ (um vector vazio).

Avaliador Exemplo

Formato de input:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de output:

```
S
R[0] R[1] ... R[S-1]
```

Aqui, S é o comprimento do vector R devolvido por `max_coupons`.